

Anatomía de un Programa en C

```
// 0. Librerías estándar en C
#include <stdint.h>
#include "peripherals.h"
```

```
// 1. Macros y constantes
#define BUFFER_SIZE 128
```

```
// 2. Definición de tipos (struct) y alias
typedef struct Message {
    uint16_t length;           // 2 bytes en ATmega328P
    uint8_t data[BUFFER_SIZE]; // 128 bytes en .bss
} Message;
```

```
// 3. Prototipos de funciones (declaraciones)
void    init_system(void);

int     send_message(const Message *msg);

extern int UART_Transmit(const uint8_t *data, uint16_t len);
```

```
// 4. Programa Principal
int main(void)
{
    init_system();           // Inicializar hardware

    Message msg = { .length = 0, .data = {0} }; // Stack: ~2+128 bytes

    while (1) {
        if (send_message(&msg) < 0) {
            // Manejo de error
        }
    }
    return 0;                // Nunca se llega
}
```

```
// 5. Definición de funciones en C
void init_system(void)
{
    // Configuración de GPIO, timers, UART...
}

int send_message(const Message *msg)
{
    // Llamada a función en Assembly
    return UART_Transmit(msg->data, msg->length);
}
```

Notas:

- `#include <...>` recurre al path del compilador
- Un **fichero** (o unidad de traducción) es cada `.c`, `.h` o `.s` que compila el linker.
- `extern` declara símbolos externos (funciones o variables) definidos en otro fichero.

Tipado Estático y Tamaño en Memoria (ATmega328P)

Tipo	Código	Tamaño en bytes	Equivalente Python
Entero 16b	<code>int</code>	2	<code>int</code>
Entero 8b	<code>int8_t</code>	1	<code>int</code>
Entero 16b	<code>int16_t</code>	2	<code>int</code>
Entero 32b	<code>int32_t</code>	4	<code>int</code>
Sin signo 8b	<code>uint8_t</code>	1	<code>int</code>
Sin signo 16b	<code>uint16_t</code>	2	<code>int</code>
Carácter	<code>char</code>	1	<code>str</code> (longitud 1)
Cadenas	<code>char *</code>	2 (puntero)*	<code>str</code>
Flotante	<code>float</code>	4	<code>float</code>
Doble	<code>double</code>	4**	<code>float</code>
Punteros	<code>uint8_t *</code>	2	<code>None</code>

* El puntero ocupa 2 bytes en memoria, pero la cadena apuntada puede ocupar más en `.data` o ubicaciones de flash.

** En AVR-GCC (el compilador) para ATmega328P, `double` está mapeado a precisión simple (4 bytes), igual que `float`.

Ejemplo:

```
uint8_t a = 5;           // ocupa 1 B en RAM
uint16_t b = 1000;       // ocupa 2 B en RAM
int32_t c = -1;          // ocupa 4 B en RAM
char ch = 'A';           // ocupa 1 B en RAM
char *s = "Hola";        // s ocupa 2 B (puntero). "Hola" está en flash
float f = 3.14f;          // ocupa 4 B en RAM
double d = 2.718;         // ocupa 4 B en RAM (AVR-GCC)
```

Punteros y Acceso a Memoria

Los punteros almacenan direcciones de memoria. Son esenciales para:

1. Modificar datos fuera de la función.
2. Acceder a memoria mapeada de periféricos.

```
int value = 42;           // Variable en memoria
int *p = &value;          // p contiene la dirección de 'value'

*p = 100;                 // escribe 100 en la dirección de 'value'
                          // Tras esto, value == 100
```

Tipos de Argumentos y Paso por Valor

En C todos los argumentos se pasan por valor. Para alterar variables externas a una función se usan **punteros**:

```
void increment(int *p)
{
    (*p)++; // Desreferencia y suma 1
}

int main(void)
{
    int a = 0;
    increment(&a); // Paso la dirección de 'a'
    // Ahora a == 1
    return 1
}
```

Analogía Python: en Python todo se pasa por referencia, pero C obliga a punteros.

Manejo de Memoria

Flash (Memoria de Programa)

- **Qué es:** memoria no volátil donde se almacena el **código compilado** (instrucciones) y las constantes inicializadas.
- **Tamaño:** 32 768 bytes (32 KB).
- **Acceso:** el CPU lee directamente desde Flash para ejecutar instrucciones. No se puede escribir en ella durante la ejecución normal (solo mediante un programador o la rutina de «bootloader»).
- **Segmentos relacionados:**
 - **.text:** contiene el código compilado del programa.
 - **.rodata:** datos constantes (cadenas literales, tablas const).

RAM (Memoria de Datos)

- **Qué es:** memoria volátil usada en tiempo de ejecución para:
 - Variables globales inicializadas y no inicializadas.
 - Pila (stack) de llamadas y variables locales automáticas.
 - Heap (si usas malloc).
- **Tamaño:** 2 048 bytes (2 KB).
- **Segmentos relacionados:**
 - **.data:** variables globales **inicializadas** (por ejemplo, `int x = 5;`). En arranque, el bootloader copia su valor inicial desde Flash a RAM.
 - **.bss:** variables globales **sin inicializar** (por ejemplo, `int y;`). Se rellenan a cero en arranque.

Alcance de Variables (Scope)

- **Global:** definidas fuera de cualquier función; viven en la sección `.data` o `.bss`.
- **Static (archivo):** `static int count;` solo visible dentro de ese fichero `.c`.
- **Local:** definidas en funciones o bloques `{ }`; viven en la pila (stack).

```
static int file_only; // Solo en este fichero, en .data

void foo(void)
{
    int i = 0;          // Solo en foo, en el stack
    {
        int j = 1;      // Solo en este bloque interior, en el stack
    }
    static int i = 0;    // Solo en foo, en .data
}
```

¿Qué es malloc?

malloc (memory allocation) es una función de la biblioteca estándar de C (`<stdlib.h>`) que **reserva un bloque de memoria en tiempo de ejecución (heap)** y devuelve un puntero al inicio de ese bloque.

```
#include <stdlib.h>
```

```
void* malloc(size_t size);
```

¿Qué pasa internamente?

Cuando se llama a `malloc`, el sistema:

1. Busca un bloque libre de tamaño suficiente en el **heap**.
2. Marca ese bloque como “ocupado”.
3. Devuelve un puntero a ese bloque.

La memoria no está inicializada (tiene basura). Si se quiere memoria en cero, se usa `calloc`.

¡CUIDADO!

Aunque `malloc` **existe** en `avr-libc`, **no se recomienda usarlo** en microcontroladores pequeños como el **ATmega328P** por estas razones:

Problema	Detalle
Memoria muy limitada	Solo 2 KB de SRAM (RAM), fácilmente se agota.
Fragmentación	<code>malloc</code> y <code>free</code> pueden dejar pequeños bloques sueltos inutilizables.
Impredecible	En sistemas embebidos, es clave tener control total del uso de memoria. <code>malloc</code> no lo permite.
Overhead	Internamente requiere metadatos para gestionar los bloques, que consumen memoria.

*Alternativas a **malloc***

- **Arreglos estáticos**
- **Asignación manual en RAM:** reservar regiones de memoria uno mismo.

```
static uint8_t buffer_private[64]; // global privado, en .data
uint8_t buffer_public[64]; // global publico, en .data

void myFunction(void)
{
    uint8_t buffer_local[64]; // local, en el Stack. No recomendable
    static uint8_t buffer_persistent[64]; // local persistente,
    en .data
}
```

Resumen de segmentos en tiempo de compilación/link

Segmento	Contenido	Lugar físico
<code>.text</code>	Código máquina (tus funciones)	Flash
<code>.rodata</code>	Datos constantes (<code>const</code> , literales)	Flash
<code>.data</code>	Variables globales inicializadas	RAM (copiado desde Flash)
<code>.bss</code>	Variables globales no inicializadas (cero por defecto)	RAM
Stack	Variables locales, direcciones de retorno, contextos	RAM
Heap	Asignaciones dinámicas (<code>malloc</code>)	RAM

Constantes y Calificadores: ***const*** vs ***volatile***

const

- **Qué hace:** Indica al compilador que esa variable no debe cambiarse después de su inicialización.
- **Cuándo usarlo:**
 - Para valores de configuración (baud rates, tamaños de buffer).
 - Para protecciones de API: si pasas datos a una función que no debe modificar, declara el parámetro como `const`.
- **Efecto en memoria:**
 - El compilador puede colocarlos en memoria de solo lectura (flash) cuando el micro lo permita.
 - Previene errores accidentales de escritura en tiempo de compilación.

```
const uint16_t BAUD_RATE = 9600; //constante en memoria de programa
```

volatile

- **Qué hace:** Le dice al optimizador “nunca elimines ni combines lecturas/escrituras a esta dirección, porque su valor puede cambiar fuera del control del programa”.
- **Cuándo usarlo:**
 - Registros de periféricos mapeados en memoria.
 - Variables compartidas con ISRs (flags de interrupción).
 - Buffers DMA o ubicaciones que el hardware actualiza.
- **Efecto en compilación:**
 - Cada acceso se traduce a una instrucción de lectura o escritura al objeto, garantizando sincronía con el hardware.

```
// Una bandera compartida entre ISR y main.  
// Debe ser volatile para forzar lectura en cada acceso.  
volatile uint8_t timeout_flag = 0;
```

Operaciones en C

Operaciones de Bit

En C, operar bits permite controlar registros de hardware con precisión:

Operación	Simb.	Ejemplo	Resultado
AND bit a bit	<code>&</code>	<code>0b1100 & 0b1010</code>	<code>0b1000</code>
OR bit a bit	<code> </code>	<code>0b1100 0b1010</code>	<code>0b1110</code>
XOR	<code>^</code>	<code>0b1100 ^ 0b1010</code>	<code>0b0110</code>
NOT	<code>~</code>	<code>~0b1100</code>	<code>0b0011</code>
Shift izq	<code><<</code>	<code>0b00001100 << 1</code>	<code>0b00011000</code>
Shift der	<code>>></code>	<code>0b00001100 >> 1</code>	<code>0b00000110</code>

Operaciones de Asignación

Operador	Equivalente	Ejemplo	Resultado
<code>=</code>	<code>—</code>	<code>a = 5;</code>	<code>a = 5</code>
<code>+=</code>	<code>a = a + b</code>	<code>a = 3; a += 2;</code>	<code>a = 5</code>
<code>-=</code>	<code>a = a - b</code>	<code>a = 5; a -= 2;</code>	<code>a = 3</code>
<code>*=</code>	<code>a = a * b</code>	<code>a = 4; a *= 3;</code>	<code>a = 12</code>
<code>/=</code>	<code>a = a / b</code>	<code>a = 10; a /= 4;</code>	<code>a = 2</code>
<code>%=</code>	<code>a = a % b</code>	<code>a = 10; a %= 4;</code>	<code>a = 2</code>
<code><<=</code>	<code>a = a << b</code>	<code>a = 1; a <<= 3;</code>	<code>a = 8</code>
<code>>>=</code>	<code>a = a >> b</code>	<code>a = 8; a >>= 2;</code>	<code>a = 2</code>
<code>&=</code>	<code>a = a & b</code>	<code>a = 0xFF; a &= 3;</code>	<code>a = 3</code>
<code>^=</code>	<code>a = a ^ b</code>	<code>a = 5; a ^= 3;</code>	<code>a = 6</code>
<code> </code>	<code>a = a b</code>	<code>a = 5; a = 3;</code>	<code>a = 6</code>

Ejemplos:

```
uint8_t flags = 0;

// Encender bit 3
flags |= (1 << 3);

// Apagar bit 1
flags &= ~(1 << 1);

// Comprobar bit 2
if (flags & (1 << 2)) {
    // true si bit 2 está en 1
}

// Invertir bit 0
flags ^= (1 << 0);

// Macros para manipular bits
#define SET_BIT(reg, bit)      ((reg) |= (1U << (bit)))
#define CLEAR_BIT(reg, bit)   ((reg) &= ~(1U << (bit)))
#define TOGGLE_BIT(reg, bit)  ((reg) ^= (1U << (bit)))
#define READ_BIT(reg, bit)    (((reg) >> (bit)) & 1U)
#define WRITE_BIT(reg, bit, v) \
    do { if (v) SET_BIT(reg, bit); else CLEAR_BIT(reg, bit); } while \
    (0)
```

Ejemplo de uso en ATmega328P para el pin PB5:

```
// Configuro PB5 como salida
DDRB |= (1 << PB5);

// Enciendo el LED
SET_BIT(PORTB, PB5);

// Apago el LED
CLEAR_BIT(PORTB, PB5);

// Alterno el LED
TOGGLE_BIT(PORTB, PB5);

// Leo el estado de PB5
uint8_t estado = READ_BIT(PINB, PB5);
```